Connection Pooling

Connection pooling is a technique where an application keeps a **pool (cache) of pre-established connections** (usually to a database) and **reuses them** instead of creating a new connection for every request.

How It Works

When an application needs to interact with a database, instead of opening a new connection each time, it:

1. **Requests a connection** from the pool
2. **Uses the connection** to execute queries
3. **Returns the connection** to the pool (rather than closing it)
4. The connection stays alive and **ready for reuse**

Why do we need it?

Creating a database connection is **expensive**:

- TCP handshake
- Authentication
- Memory allocation
- Network latency

If every request opened and closed its own connection:

- ❌ Slow response times
- ❌ High CPU & memory usage
- ❌ Database overload

Key Concepts

Pool Size: The number of connections maintained in the pool. This typically includes:

Minimum connections: Always kept alive

Maximum connections: Upper limit the pool can grow to

Idle connections: Unused but available connections

Connection Lifecycle: Connections can be in different states—active (in use), idle (available), or stale (needs validation/renewal).

Timeout Settings: Controls like connection timeout (how long to wait for an available connection) and idle timeout (when to close unused connections).

How it works (step-by-step)

1. App starts → creates a **fixed number of DB connections**
2. A request comes in:
    - Gets an **available connection** from the pool
3. Query executes
4. Connection is **returned to the pool** (not closed!)
5. Next request reuses it

If no connection is available:

- Request **waits**
- Or fails after a timeout

Key Pool Configuration Parameters

Parameter	Meaning
`maxPoolSize`	Max connections allowed
`minIdle`	Minimum idle connections
`connectionTimeout`	How long to wait for a connection
`idleTimeout`	When idle connections are closed
`maxLifetime`	Max age of a connection

Common Connection Pool Libraries

Java / Spring Boot

- **HikariCP** (default, fastest 🚀)
- Apache DBCP
- C3P0